

W7D4 – Esercitazione Python AI

Studente: Riccardo Luzzi

Corso: Cyber Security

Sistema utilizzato: Kali Linux

Ambiente di sviluppo: Visual Studio Code

Linguaggio: Python

Introduzione

In questa esercitazione ho continuato a lavorare con Python e le API di Google Gemini, riprendendo l'ambiente configurato nella precedente attività. Questa volta ho svolto tre prove: l'analisi di un file di log con Pandas e Gemini, una chiamata semplice con `hello_gemini.py` e infine la realizzazione di una chatbot utilizzabile direttamente dal terminale.

Anche in questo caso ho preferito mantenere gli esercizi in file separati. In questo modo ho potuto provare ogni parte singolarmente e controllare più facilmente il risultato ottenuto.

1. Preparazione dell'ambiente di lavoro

Per questa esercitazione ho creato una nuova cartella di lavoro, separata da quella utilizzata per W7D1. All'interno della nuova cartella ho creato un nuovo ambiente virtuale chiamato `env` e i tre file Python utilizzati per gli esercizi: `pandas_gemini.py`, `hello_gemini.py` e `chatbot.py`. Successivamente ho aggiunto anche `chatbot1.py` per provare una seconda versione della chatbot con memoria della conversazione.

Dopo aver creato il nuovo ambiente ho installato le librerie necessarie e l'ho attivato dal terminale. Nella stessa cartella ho inserito anche il file `auth.log.csv` necessario per l'esercizio con Pandas, oltre al file `.env` contenente la variabile `GEMINI_API_KEY` e al file `requirements.txt` con le dipendenze utilizzate nel progetto.

- **`source ./env/bin/activate`**

La chiave API viene caricata tramite `python-dotenv`, evitando quindi di inserirla direttamente nel codice.

- **`load_dotenv()`**
- **`api_key = os.environ["GEMINI_API_KEY"]`**

2. Analisi dei log con Pandas e Gemini

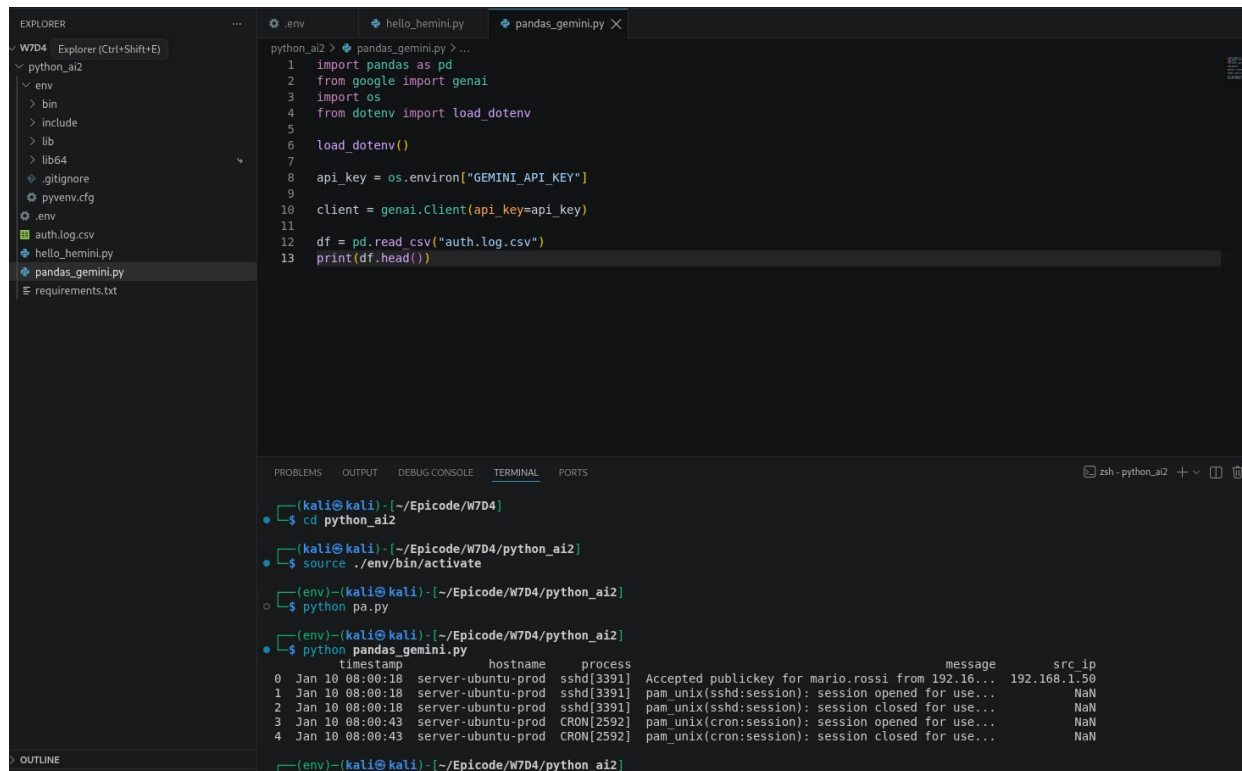
2.1 Lettura del file CSV

Il primo esercizio richiedeva di analizzare un file di log di autenticazione. Ho copiato auth.log.csv nella stessa cartella dello script pandas_gemini.py e l'ho aperto con Pandas utilizzando read_csv().

- `df = pd.read_csv("auth.log.csv")`

Come prima verifica ho stampato le prime righe del DataFrame con df.head(). In questo modo ho controllato che il file venisse letto correttamente e ho visto le colonne disponibili, tra cui timestamp, hostname, process, message e src_ip.

Figura 1 – Lettura del file auth.log.csv con Pandas



The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'W7D4' with a folder 'python_ai2' containing files like '.env', 'auth.log.csv', 'hello_gemini.py', 'pandas_gemini.py', and 'requirements.txt'. The code editor shows the 'pandas_gemini.py' file with the following code:

```
python_ai2 > pandas_gemini.py > ...
1 import pandas as pd
2 from google import genai
3 import os
4 from dotenv import load_dotenv
5
6 load_dotenv()
7
8 api_key = os.environ["GEMINI_API_KEY"]
9
10 client = genai.Client(api_key=api_key)
11
12 df = pd.read_csv("auth.log.csv")
13 print(df.head())
```

The terminal output shows the execution of the script, including the command 'python pandas_gemini.py' and the resulting DataFrame head:

```
(env)-(kali@kali)-[~/Epicode/W7D4/python_ai2]
$ python pandas_gemini.py
timestamp      hostname      process      message      src_ip
0 Jan 10 08:00:18 server-ubuntu-prod sshd[3391] Accepted publickey for mario.rossi from 192.168.1.50
1 Jan 10 08:00:18 server-ubuntu-prod sshd[3391] pam_unix(sshd:session): session opened for use...
2 Jan 10 08:00:18 server-ubuntu-prod sshd[3391] pam_unix(sshd:session): session closed for use...
3 Jan 10 08:00:43 server-ubuntu-prod CRON[2592] pam_unix(cron:session): session opened for use...
4 Jan 10 08:00:43 server-ubuntu-prod CRON[2592] pam_unix(cron:session): session closed for use...
```

2.2 Filtro degli eventi FAILED

Dopo aver verificato il contenuto del CSV ho filtrato la colonna message cercando la parola “Failed”. Ho utilizzato str.contains() con case=False per non distinguere tra maiuscole e minuscole e na=False per evitare problemi con eventuali valori mancanti.

- `df["message"].str.contains("Failed", case=False, na=False)`

Con head(20) ho limitato il risultato ai primi venti eventi. Nel terminale sono comparsi diversi tentativi di autenticazione falliti, molti dei quali provenienti dallo stesso indirizzo IP.

Figura 2 – Filtro dei tentativi di autenticazione falliti

```
python_ai2 > pandas_gemini.py >...
1  import pandas as pd
2  from google import genai
3  import os
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  api_key = os.environ["GEMINI_API_KEY"]
9
10 client = genai.Client(api_key=api_key)
11
12 df = pd.read_csv("auth.log.csv")
13
14 suspicious = df[
15     df["message"].str.contains("Failed", case=False, na=False)
16 ].head(20)
17
18 print(suspicious)
```



```
(env)-(kali@kali) - [~/Epicode/W7D4/python_ai2]
$ python pandas_gemini.py
41 Jan 10 08:03:54 server-ubuntu-prod sshd[8513] Failed password for invalid user postgres from... 46.59.23.16
45 Jan 10 08:03:56 server-ubuntu-prod sshd[9342] Failed password for root from 46.59.23.16 port... 46.59.23.16
50 Jan 10 08:03:57 server-ubuntu-prod sshd[9017] Failed password for invalid user root from 46... 46.59.23.16
53 Jan 10 08:03:57 server-ubuntu-prod sshd[4814] Failed password for invalid user root from 46... 46.59.23.16
58 Jan 10 08:03:59 server-ubuntu-prod sshd[8434] Failed password for invalid user postgres from... 46.59.23.16
61 Jan 10 08:04:01 server-ubuntu-prod sshd[4028] Failed password for invalid user test from 46... 46.59.23.16
64 Jan 10 08:04:04 server-ubuntu-prod sshd[7660] Failed password for root from 46.59.23.16 port... 46.59.23.16
67 Jan 10 08:04:04 server-ubuntu-prod sshd[7613] Failed password for invalid user user from 46... 46.59.23.16
70 Jan 10 08:04:06 server-ubuntu-prod sshd[8561] Failed password for invalid user postgres from... 46.59.23.16
71 Jan 10 08:04:07 server-ubuntu-prod sshd[5828] Failed password for root from 46.59.23.16 port... 46.59.23.16
74 Jan 10 08:04:07 server-ubuntu-prod sshd[2105] Failed password for invalid user root from 46... 46.59.23.16
77 Jan 10 08:04:09 server-ubuntu-prod sshd[9364] Failed password for invalid user user from 46... 46.59.23.16
78 Jan 10 08:04:10 server-ubuntu-prod sshd[6371] Failed password for root from 46.59.23.16 port... 46.59.23.16
```

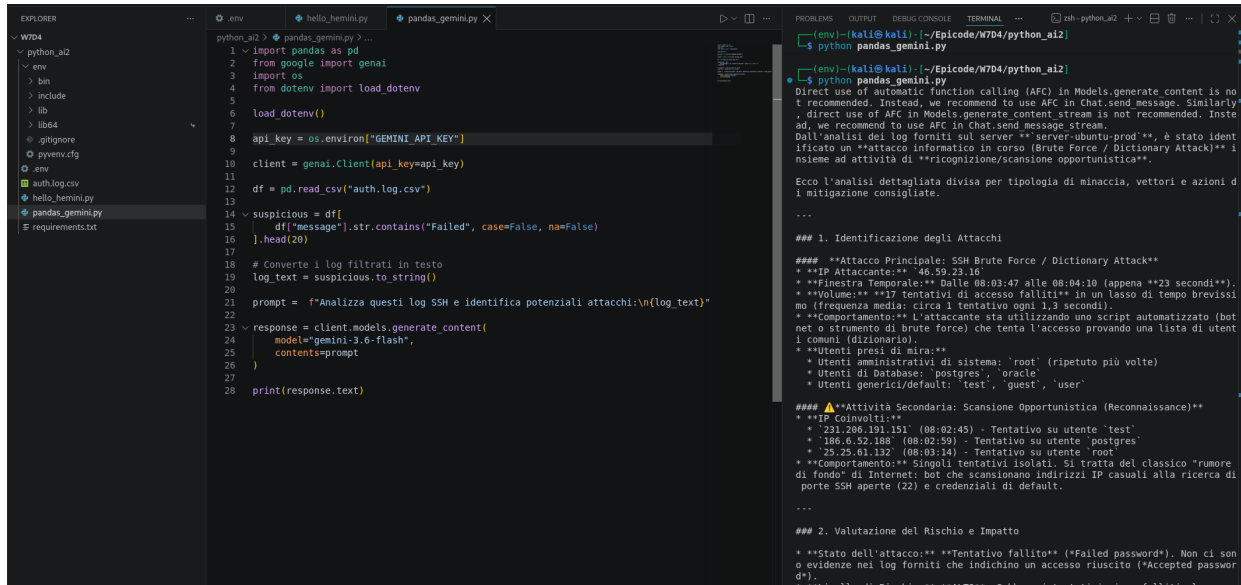
2.3 Invio dei log a Gemini

Una volta ottenuti gli eventi interessanti, ho convertito il DataFrame filtrato in una stringa tramite `to_string()`. Ho poi inserito il testo dei log all'interno di un prompt chiedendo a Gemini di analizzare gli eventi SSH e individuare possibili attacchi.

- `log_text = suspicious.to_string()`
- `prompt = f"Analizza questi log SSH e identifica potenziali attacchi:\n{log_text}"`

Il prompt viene passato a `generate_content()` e la risposta viene stampata nel terminale. Gemini ha riconosciuto una sequenza compatibile con tentativi di brute force/dictionary attack e ha distinto anche alcuni tentativi isolati provenienti da altri indirizzi.

Figura 3 – Analisi dei log filtrati tramite Gemini



```
python_ai2 > python pandas_gemini.py ...
1 import pandas as pd
2 from google import generai
3 import os
4 from dotenv import load_dotenv
5 load_dotenv()
6
7
8 api_key = os.environ["GEMINI_API_KEY"]
9
10 client = generai.Client(api_key=api_key)
11
12 df = pd.read_csv("auth.log.csv")
13
14 suspicious = df[
15     df["message"].str.contains("Failed", case=False, na=False)
16 ].head(20)
17
18 # Converti i log filtrati in testo
19 log_text = suspicious.to_string()
20
21 prompt = f"Analizza questi log SSH e identifica potenziali attacchi:\n(log_text)"
22
23 response = client.models.generate_content(
24     model="gemini-3.6-flash",
25     contents=prompt
26 )
27
28 print(response.text)
```

```
(env)-(kali@kali) - [~/Epicode/W704/python_ai2]
$ python pandas_gemini.py

Direct use of automatic function calling (AFC) in Models.generate content is not recommended. Instead, we recommend to use AFC in Chat.send message. Similarly, direct use of AFC in Models.generate content stream is not recommended. Instead, we recommend to use AFC in Chat.send message stream.
Dall'analisi dei log forniti sul server "server-ubuntu-prod", è stato identificato un "attacco informatico in corso (Brute Force / Dictionary Attack)" insieme ad attività di "ricognizione/scansione opportunistica".

Ecco l'analisi dettagliata divisa per tipologia di minaccia, vettori e azioni di mitigazione consigliate.

...

### 1. Identificazione degli Attacchi

#### **Attacco Principale: SSH Brute Force / Dictionary Attack**
* **IP Attaccante:** 46.59.23.16
* **Finestra Temporale:** Dalle 08:03:47 alle 08:04:10 (appena **23 secondi**).
* **Volume:** **17 tentativi di accesso falliti** in un lasso di tempo brevissimo (frequenza media: circa 1 tentativo ogni 1,3 secondi).
* **Comportamento:** L'attaccante sta utilizzando uno script automatizzato (bot net o strumento di brute force) che tenta l'accesso provando una lista di utenti comuni (dizionario).
  * **Utenti presi di mira:**
    * Utenti amministrativi di sistema: 'root' (ripetuto più volte)
    * Utenti di Database: 'postgres', 'oracle'
    * Utenti generici/default: 'test', 'guest', 'user'

#### **Attività Secondaria: Scansione Opportunistica (Reconnaissance)**
* **IP Coinvolti:**
  * 221.206.191.151 (08:02:45) - Tentativo su utente 'test'
  * 186.6.52.188 (08:02:59) - Tentativo su utente 'postgres'
  * 25.25.61.132 (08:03:14) - Tentativo su utente 'root'
* **Comportamento:** Singoli tentativi isolati. Si tratta del classico "rumore di fondo" di Internet: bot che scansionano indirizzi IP casuali alla ricerca di porte SSH aperte (22) e credenziali di default.

...

### 2. Valutazione del Rischio e Impatto

* **Stato dell'attacco:** Tentativo fallito (**Failed password**). Non ci sono evidenze nei log forniti che indichino un accesso riuscito (**Accepted password**).
* **Rischio di Rischio:** Sebbene i tentativi siano falliti, la natura dell'attacco (SSH Brute Force) è potenzialmente dannosa se non viene mitigata.
```

3. Hello Gemini

3.1 File hello_gemini.py

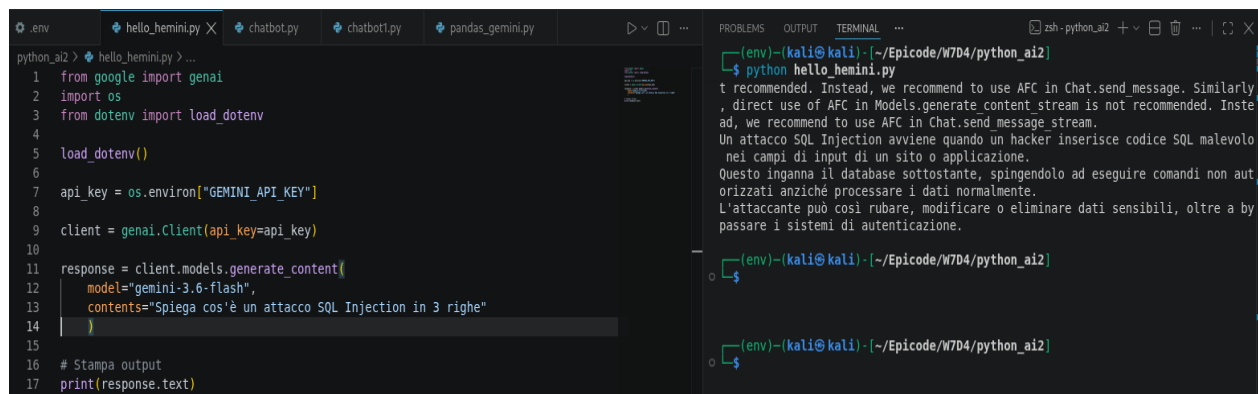
Nel secondo esercizio ho realizzato una chiamata semplice a Gemini. Lo scopo era verificare nuovamente il funzionamento del client e chiedere al modello di spiegare che cos'è un attacco SQL Injection in tre righe.

- **contents="Spiega cos'è un attacco SQL Injection in 3 righe"**

La risposta viene recuperata con response.text e stampata nel terminale. Il programma ha funzionato correttamente e ha restituito una spiegazione breve, rispettando la richiesta indicata nel prompt.

Questo esercizio è più semplice rispetto a quello con Pandas perché non è necessario elaborare dati prima della chiamata: il testo viene inviato direttamente al modello e la risposta viene mostrata a video.

Figura 4 – Chiamata semplice a Gemini



```
python_ai2 > python hello_gemini.py ...
1 from google import generai
2 import os
3 from dotenv import load_dotenv
4 load_dotenv()
5
6
7 api_key = os.environ["GEMINI_API_KEY"]
8
9 client = generai.Client(api_key=api_key)
10
11 response = client.models.generate_content(
12     model="gemini-3.6-flash",
13     contents="Spiega cos'è un attacco SQL Injection in 3 righe"
14 )
15
16 # Stampa output
17 print(response.text)
```

```
(env)-(kali@kali) - [~/Epicode/W704/python_ai2]
$ python hello_gemini.py

Direct use of automatic function calling (AFC) in Chat.send message. Similarly, direct use of AFC in Models.generate content stream is not recommended. Instead, we recommend to use AFC in Chat.send message stream.
Un attacco SQL Injection avviene quando un hacker inserisce codice SQL malevolo nei campi di input di un sito o applicazione.
Questo inganna il database sottostante, spingendolo ad eseguire comandi non autorizzati anziché processare i dati normalmente.
L'attaccante può così rubare, modificare o eliminare dati sensibili, oltre a bypassare i sistemi di autenticazione.
```

4. Chatbot interattiva

4.1 Versione base: chatbot.py

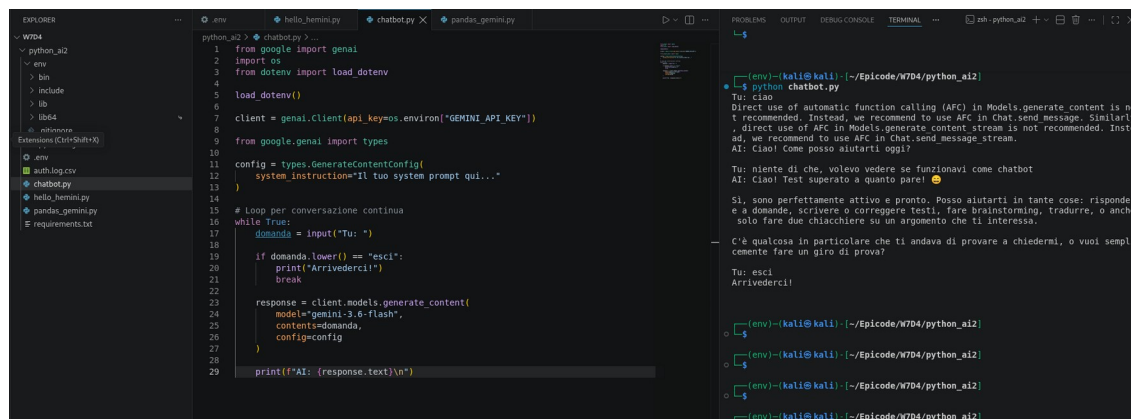
Per l'ultimo esercizio ho creato chatbot.py partendo dal codice utilizzato in precedenza. Ho aggiunto un ciclo while True che continua a chiedere un input all'utente e invia ogni domanda a Gemini.

All'interno del ciclo ho previsto anche il comando "esci". Quando viene inserito, il programma stampa "Arrivederci!" e utilizza break per terminare il ciclo.

- **domanda = input("Tu: ")**
- **if domanda.lower() == "esci":**

In questa prima versione ogni domanda viene inviata con client.models.generate_content(). Il ciclo rende quindi possibile fare più richieste senza riavviare lo script, ma le richieste rimangono separate: il modello non riceve automaticamente i messaggi precedenti come contesto.

Figura 5 – Chatbot base con ciclo di conversazione



```
python_ai2 > python chatbot.py
1 from google import genai
2 import os
3 from dotenv import load_dotenv
4
5 load_dotenv()
6
7 client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
8
9 from google.genai import types
10
11 config = types.GenerateContentConfig(
12     system_instruction="Il tuo system prompt qui..."
13 )
14
15 # loop per conversazione continua
16 while True:
17     domanda = input("Tu: ")
18
19     if domanda.lower() == "esci":
20         print("Arrivederci!")
21         break
22
23     response = client.models.generate_content(
24         model="gemini-3.6-flash",
25         contents=domanda,
26         config=config
27     )
28
29     print(f"AI: {response.text}\n")
```

```
python chatbot.py
Tu: Ciao
AI: Ciao! Come posso aiutarti oggi?
Tu: niente di che, volevo vedere se funzionavi come chatbot
AI: Ciao! Test superato a quanto pare! 😊
Si, sono perfettamente attivo e pronto. Posso aiutarti in tante cose: rispondere a domande, scrivere o correggere testi, fare brainstorming, tradurre, o anche solo fare due chiacchiere su un argomento che ti interessa.
C'è qualcosa in particolare che ti andava di provare a chiedermi, o vuoi semplicemente fare un giro di prova?
Tu: esci
Arrivederci!
```

4.2 Seconda versione con memoria della chat

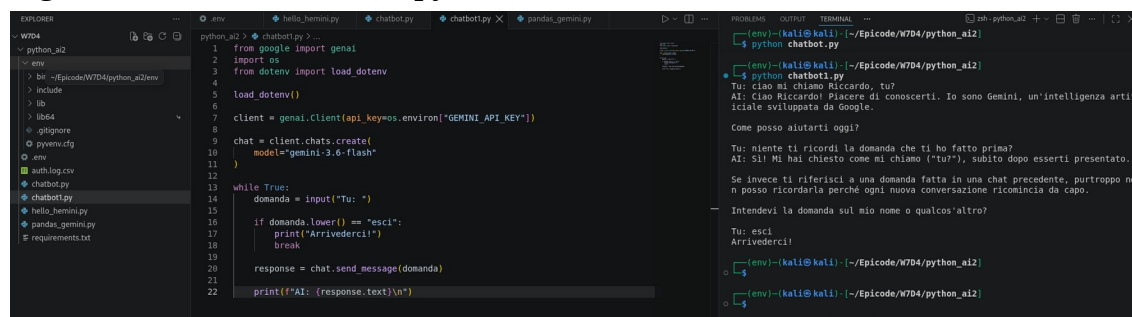
Dopo aver completato la versione richiesta, ho provato anche una seconda soluzione nel file chatbot1.py. Il motivo è che un semplice ciclo while permette di fare più domande, ma non crea da solo una vera conversazione con memoria del contesto.

In chatbot1.py ho quindi creato una sessione tramite client.chats.create() e, per ogni nuovo messaggio, ho utilizzato chat.send_message(domanda). In questo modo i messaggi della stessa sessione vengono mantenuti nel contesto della chat.

- **chat = client.chats.create(model="gemini-3.6-flash")**
- **response = chat.send_message(domanda)**

Per verificare la differenza ho scritto il mio nome nel primo messaggio e successivamente ho chiesto se ricordasse la domanda precedente. La risposta ha ripreso correttamente l'informazione fornita prima. Infine ho utilizzato "esci" per terminare il programma.

Figura 6 – Versione chatbot1.py con memoria della conversazione



The screenshot displays a code editor with a file explorer on the left and a terminal on the right. The file explorer shows a project named 'python_ai2' with subfolders 'src' and 'include', and files 'auth.log.csv', 'chatbot.py', 'chatbot1.py', 'hello_gemini.py', 'pandas_gemini.py', and 'requirements.txt'. The 'chatbot1.py' file is open in the editor, showing the following code:

```
1 from google import genai
2 import os
3 from dotenv import load_dotenv
4
5 load_dotenv()
6
7 client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
8
9 chat = client.chats.create(
10     model="gemini-1.5-flash"
11 )
12
13 while True:
14     domanda = input("Tu: ")
15
16     if domanda.lower() == "esci":
17         print("Arrivederci!")
18         break
19
20     response = chat.send_message(domanda)
21     print(f"AI: {response.text}")
```

The terminal on the right shows the execution of the script. It starts with the command `python chatbot1.py`. The output shows a conversation between a user (Tu) and an AI (AI). The user asks "Ciao mi chiamo Riccardo, tu?". The AI responds "Ciao Riccardo! Piacere di conoscerti. Io sono Gemini, un'intelligenza artificiale sviluppata da Google." The user then asks "Come posso aiutarti oggi?". The AI responds "Tu: niente ti ricordi la domanda che ti ho fatto prima?". The user then asks "Sì! Mi hai chiesto come mi chiamo ("tu?"), subito dopo esserti presentato." The AI responds "Se invece ti riferisci a una domanda fatta in una chat precedente, purtroppo non posso ricordarla perché ogni nuova conversazione ricomincia da capo." The user then asks "Intendevi la domanda sul mio nome o qualcos'altro?". The AI responds "Tu: esci". The user then types "Arrivederci!". The terminal shows the command `python chatbot1.py` being executed again, and the output shows the AI responding "Arrivederci!".

5. Conclusioni

Nel complesso l'esercitazione mi ha permesso di utilizzare Gemini in tre situazioni diverse. Con Pandas ho prima letto e filtrato un file CSV e poi ho inviato a Gemini solo gli eventi che mi interessavano. Questo passaggio mi ha fatto vedere come una libreria per l'analisi dei dati possa essere utilizzata insieme a un LLM.

Con `hello_gemini.py` ho verificato la chiamata più semplice, mentre con `chatbot.py` ho realizzato un programma che rimane in esecuzione e permette di inserire più domande dal terminale.

Ho infine provato una seconda versione della chatbot usando una sessione chat. La differenza è stata utile da vedere in pratica: il ciclo `while` da solo ripete le richieste, mentre la sessione chat permette di mantenere il contesto dei messaggi precedenti durante la stessa esecuzione.